

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №4

по теме:
АРХИТЕКТУРА SDR-УСТРОЙСТВ. ПРИМЕРЫ ФОРМИРОВАНИЯ
I/Q-СЭМПЛОВ ПРОИЗВОЛЬНОЙ ФОРМЫ. РАБОТА С БУФЕРОМ
ПРИЕМА SDR

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель:	3
2	ЛЕКЦИЯ	4
2.1	Задачи цифровой системы связи	4
2.2	Архитектура цифровой системы связи	4
2.2.1	Архитектура передатчика	4
2.3	Схемы модуляции	7
2.3.1	BPSK	7
2.3.2	QPSK	9
3	ПРАКТИКА	11
3.1	Треугольный сигнал	11
3.1.1	Программная реализация	11
3.2	Параболический сигнал	12
3.2.1	Программная реализация	12
3.3	Реализация BPSK модуляции	13
3.3.1	BPSK модулятор	13
3.3.2	Формирующий фильтр	14
3.3.3	Перемножение с несущим колебанием	14
3.4	Реализация QPSK модуляции	15
3.4.1	QPSK модулятор	15
3.4.2	Формирующий фильтр	16
3.4.3	Перемножение с несущим колебанием	17
3.5	Результат работы	18
3.5.1	Визуализация параболического сигнала до отправки ...	18
3.5.2	Визуализация параболического сигнала на приеме	18
3.5.3	Визуализация треугольного сигнала на приеме	19
4	ВЫВОД	20

ВВЕДЕНИЕ

1.1 Введение

На прошлом занятии мы формировали семплы произвольной формы с целью улучшения навыков работы с SDR с помощью C/C++ и SoapySDR. На этом занятии продолжим формировать произвольные сигналы и перейдем к работе с BPSK/QPSK модуляцией.

1.2 Цель:

Повторить архитектуру систем цифровой связи. Узнать, что такое модуляция и познакомиться с BPSK и QPSK модуляциями. Построить на языке Python модель, которая будет поэтапно визуализировать процесс модуляции.

ЛЕКЦИЯ

2.1 Задачи цифровой системы связи

Какие задачи у системы связи? Задача системы связи заключается в надежном передать поток бит на заданной скорости по каналу связи. Для передачи по каналу связи мы используем электромагнитные колебания - $\sin$ и $\cos$ какой-то частоты.

2.2 Архитектура цифровой системы связи

2.2.1 Архитектура передатчика

Базовая архитектура цифровой системы связи в простейшем случае состоит из приемника, передатчика и радиоканала. Рассмотрим упрощенную архитектуру передатчика:

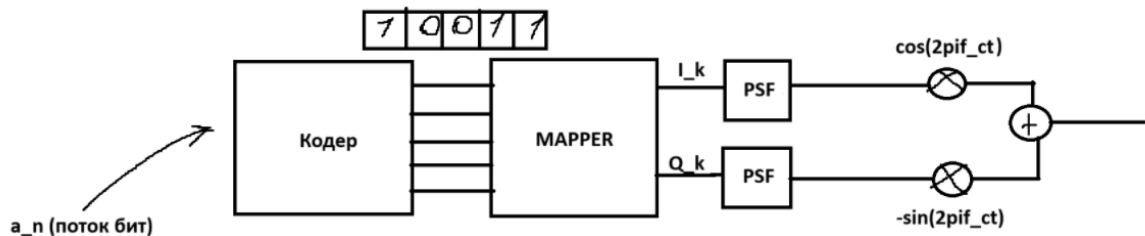


Рисунок 1 — Архитектура передатчика

Формируется поток бит, который поступает на кодер. В нашем случае кодер просто делит поток бит на блоки определенной длины. У кодера один вход, по которому последовательно поступают биты, а на выходе N-битная шина, которая уже параллельно передает биты на мапер. Кодер можно представить в виде схемы следующим образом:

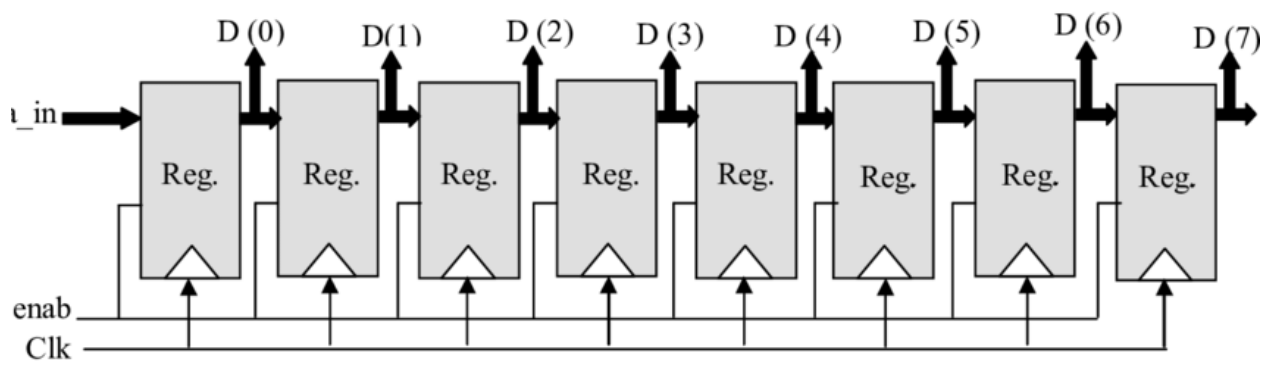


Рисунок 2 — Схема последовательно-параллельного преобразователя

Имеется N регистров с общим тактовым сигналом, которые соединены последовательно. По каждому тактовому сигналу поток бит будет "продвигаться" по тригерам и попадать на N -битную шину, т.е. параллельно выводиться из устройства.

Далее блоки битов попадают на мапер, который ставит в соответствие каждому блоку битов числа I и Q . Если блок бит имеет размерность N , то в мапере будет заложено 2^N комбинаций. На выходе маппера параллельно получим числа I и Q .

Сигнал, который мы хотим сформировать имеет вид $S_k(t) = I_k \cos(2\pi f_c t) - Q_k \sin(2\pi f_c t)$, где k - номер текущего символа, I_k , Q_k - координаты символа в сигнальной диаграмме. Эти координаты мы получаем на выходе маппера. Эти числа в будущем станут параметрами колебания.

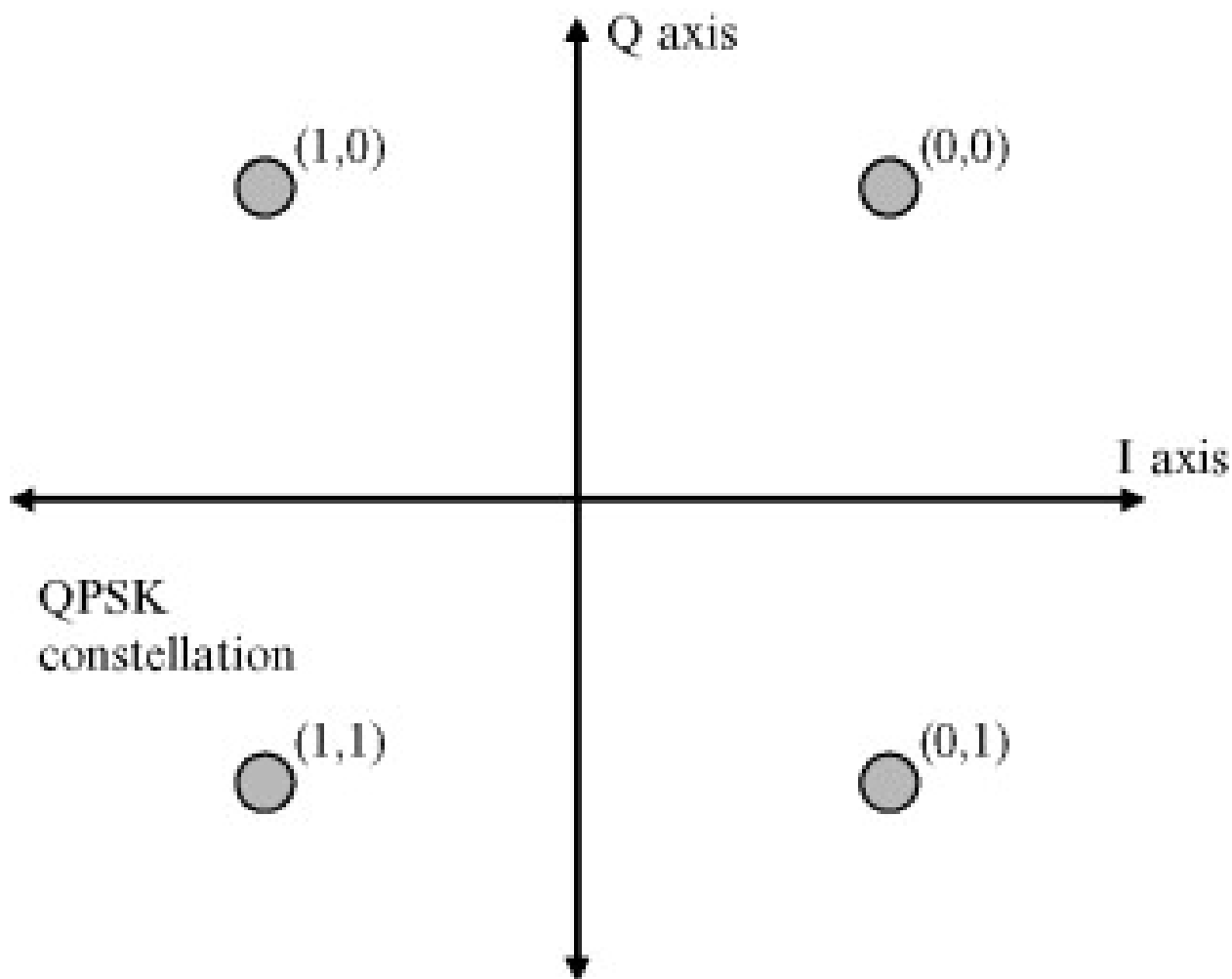


Рисунок 3 — Пример сигнальной диаграммы для QPSK модуляции

Далее I и Q попадают на Pulse Shaping Filter (PSF). Фильтр выполняет 2 задачи: определяет ширину спектра радиосигнала и его форму, а также применяется на приемной стороне для символьной синхронизации. Фильтр характеризуется важным параметром во временной области - импульсной характеристикой $g(t)$. Импульсная характеристика может быть различной, но для простоты восприятия будем рассматривать только прямоугольную характеристику. На выходе фильтра получим прямоугольные отрезки сигналов.

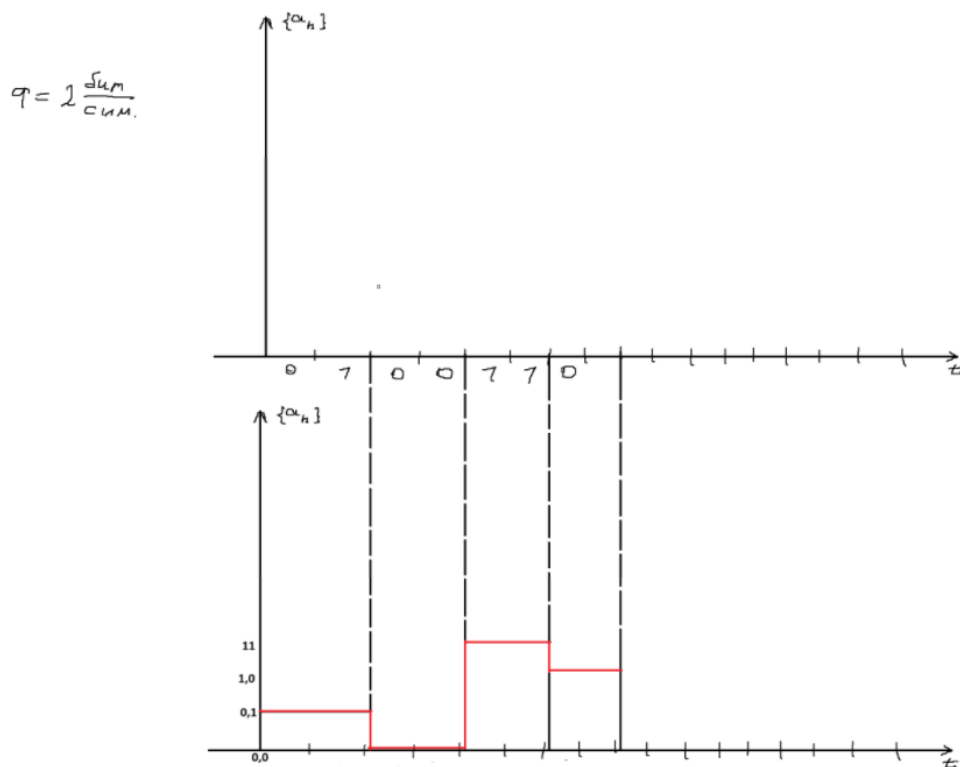


Рисунок 4 — Пример формирования символов

Длительность символов вычисляется как $\frac{1}{R_s}$, где R_s - символьная скорость. В свою очередь символьная скорость R_s вычисляется как $\frac{R_b}{\log_2(M)}$, где R_b - битовая скорость (та скорость, с которой биты поступают на маппер), M - кол-во точек созвездия. После маппера битовая скорость переходит в символьную, которая определяет длительность прямоугольного импульса.

2.3 Схемы модуляции

2.3.1 BPSK

BPSK (Binary Phase Shift Keying) - двухпозиционная фазовая манипуляция. В такой схеме модуляции на 1 бит приходится 1 символ.

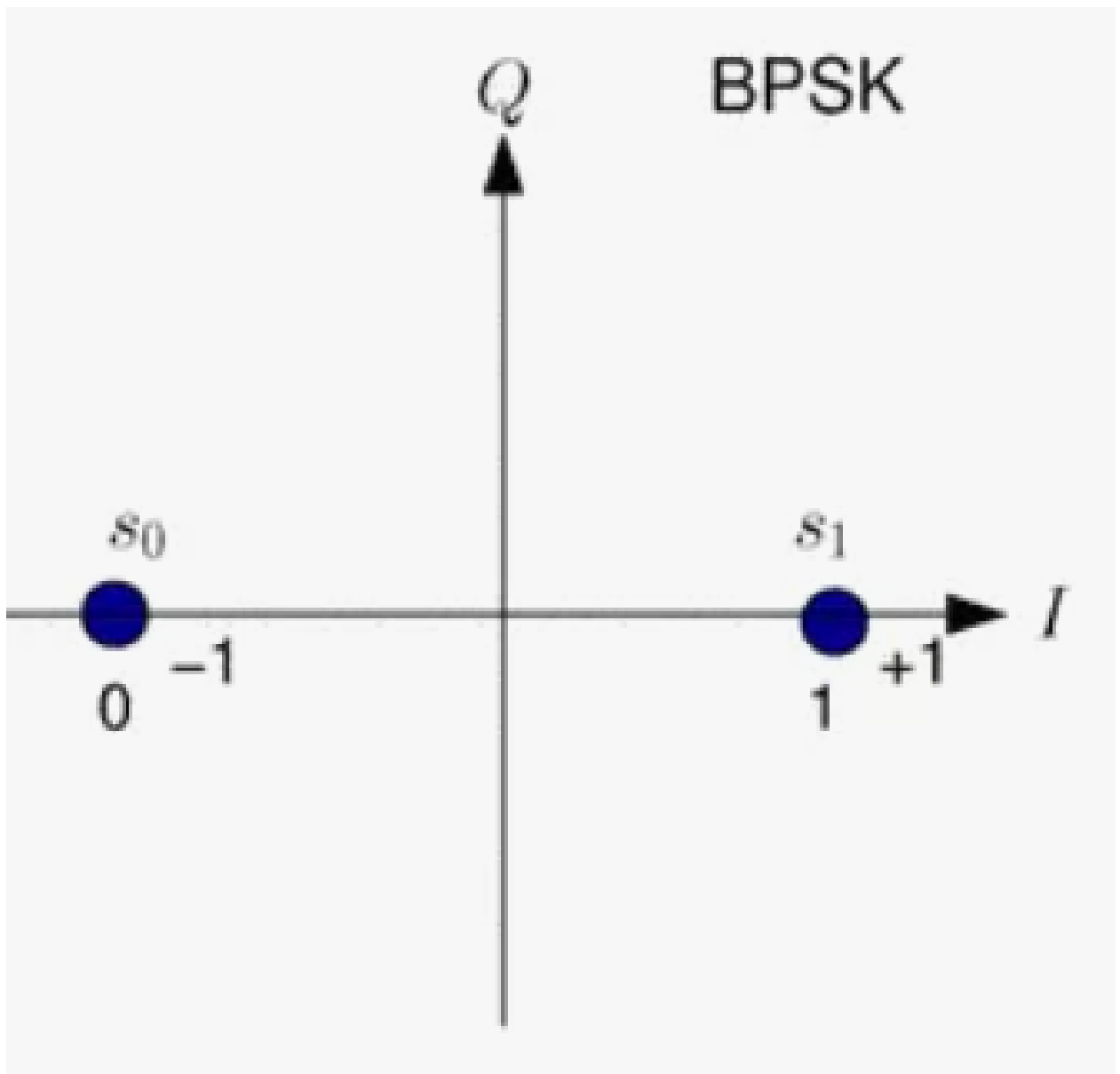


Рисунок 5 — Созвездие BPSK модуляции

Логика BPSK следующая: если бит имеет состояние 0, то он будет соответствовать сигналу с начальной фазой ϕ равной 0, т.е. точке с координатой (1, 0), если бит находится в состоянии 1, то бит будет соответствовать сигналу с начальной фазой ϕ равной π , т.е. точке с координатой (-1, 0). Можем заметить, что квадратурная составляющая всегда равна 0, это значит, что в сигнале будет только $\cos$ составляющая, т.к. $\sin$ составляющая занулится (видно из уравнения сигнала $S_k(t) = I_k \cos(2\pi f_c t) - Q_k \sin(2\pi f_c t)$).

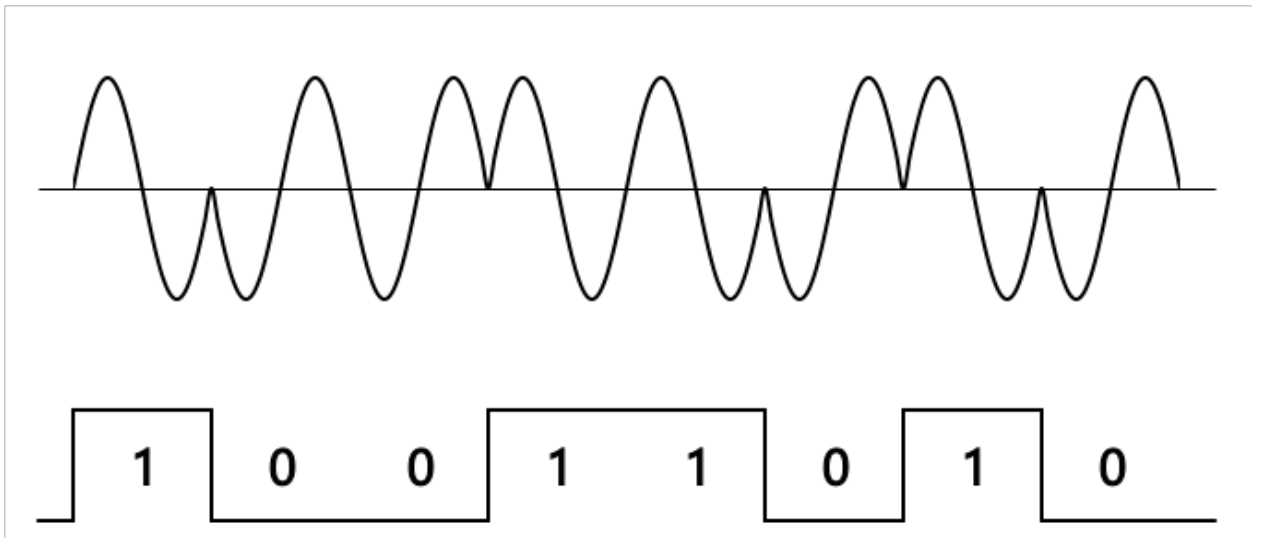


Рисунок 6 — Пример работы BPSK

2.3.2 QPSK

QPSK (Quadrature Phase Shift Keying) - квадратурная фазовая манипуляция. 2 бита кодируются в одном символе. Используется 4 различных фазы для представления пар битов. Логика этой схемы модуляции такая же, как у BPSK, но уже добавляется квадратурные составляющие, т.е в сигнале будут как $\cos$ составляющая, так и $\sin$ составляющая. Точке $(0, 0)$ будет соответствовать фаза $\frac{\pi}{4}$, $(1, 0)$ будет соответствовать фаза $\frac{3\pi}{4}$, $(1, 1)$ будет соответствовать фаза $\frac{5\pi}{4}$, $(0, 1)$ будет соответствовать фаза $\frac{7\pi}{4}$,

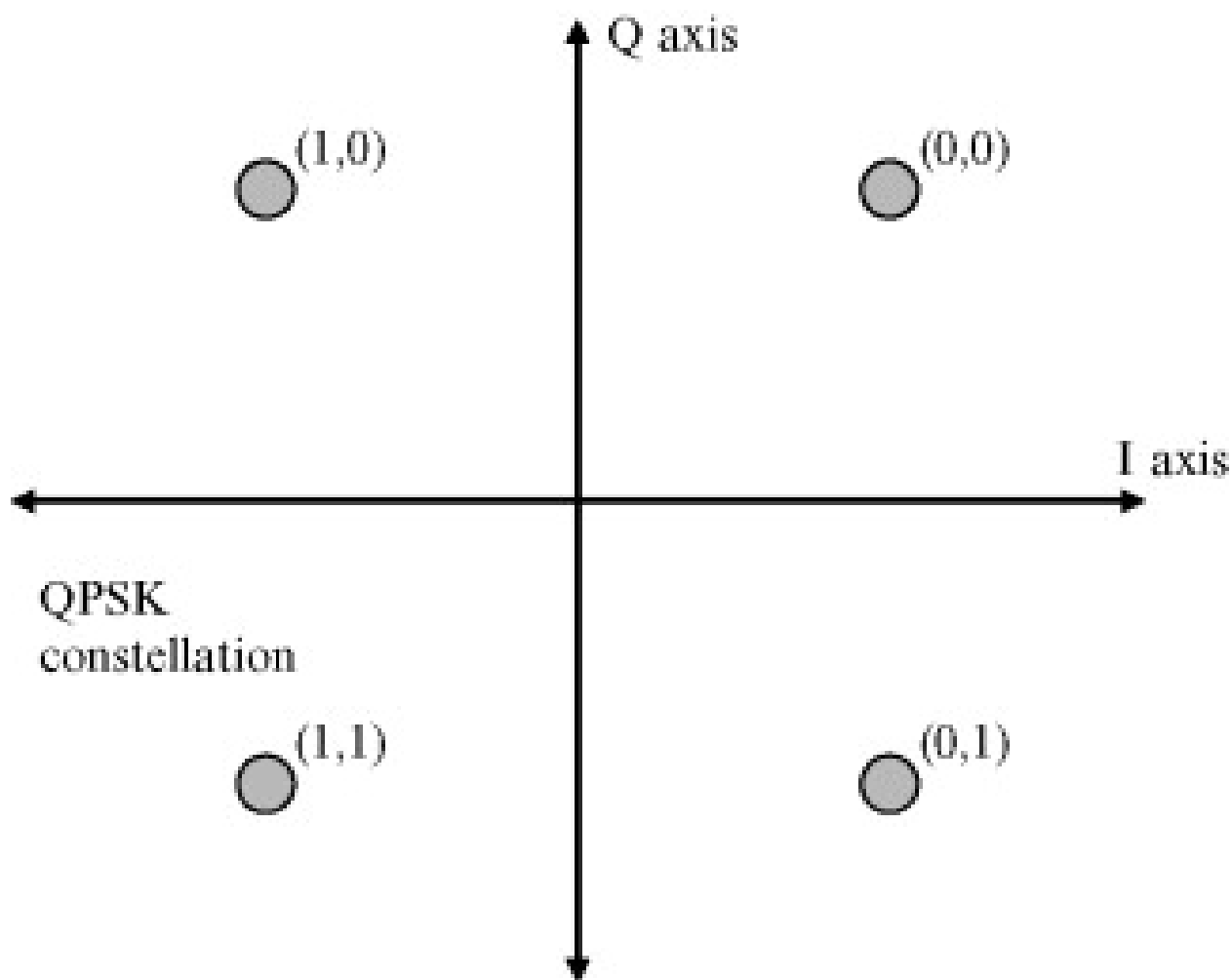


Рисунок 7 — QPSK созвездие

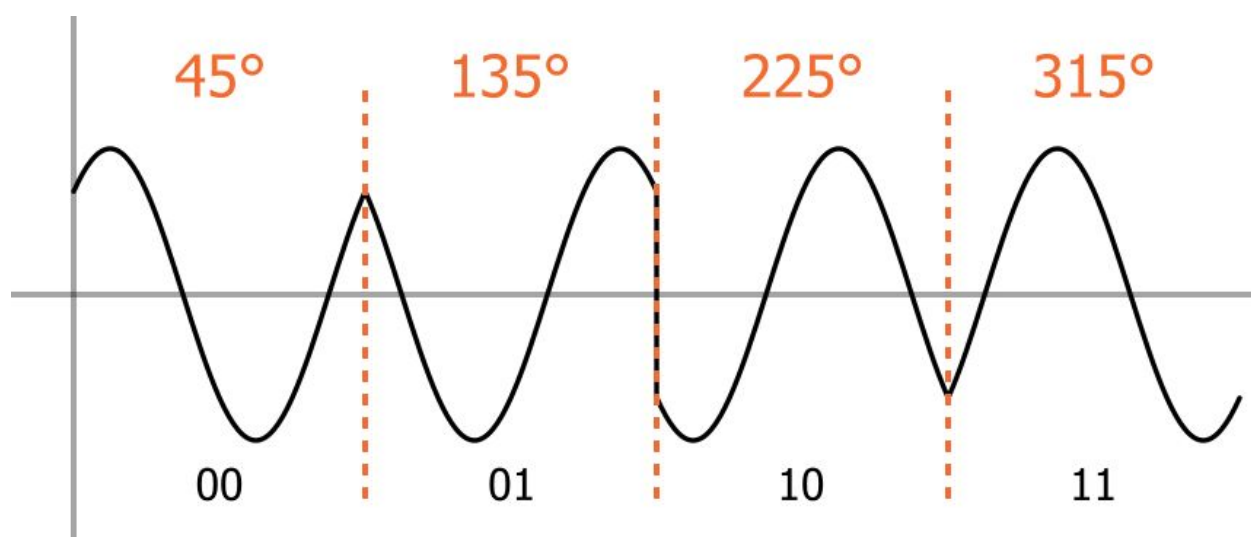


Рисунок 8 — Пример работы QPSK

ПРАКТИКА

3.1 Треугольный сигнал

Я буду формировать такой сигнал по следующему принципу:

1. Если бит равен единице, то в момент длительности сигнала $\frac{\tau}{2}$ буду задавать максимальное значение, а в остальных случаях буду задавать нули. Таким образом в одной точке будет образовываться пик, который и будет придавать сигналу форму треугольника.
2. Если бит равен 0, то на всей длительности τ сигнал будет принимать значение 0.

3.1.1 Программная реализация

```
#define TAU 10
#define TAU_ON_ELEMENT 20

int16_t* bits_to_triangle_signal(uint8_t* bits, int
    bits_count, int tx_mtu){

    // allocate memory
    int16_t* tx_buff = (int16_t*)malloc(sizeof(int16_t) * tx_mtu
        * 2);

    // iterate on bits
    for (int i = 0; i < bits_count; ++i)
    {
        // fill tx_buff with samples
        for(int j = i*TAU_ON_ELEMENT; j < i*TAU_ON_ELEMENT + 20
            && j < tx_mtu*2; j+=2){
            if(bits[i] && j % 10 == 5){
                tx_buff[j] = 2047 << 4;    // I
                tx_buff[j+1] = -2047 << 4; // Q
            } else{
                tx_buff[j] = 0;           //I
                tx_buff[j+1] = 0;         //Q
            }
        }
    }
}
```

```

    }
}

return tx_buff;
}

```

Код полностью идентичен коду, который использовался для создания прямоугольного сигнала. Единственное отличие заключается в условии $j \% 10 == 5$, которое позволяет выбирать середины символов.

3.2 Параболический сигнал

Сформируем параболу, перебирая точки от $-\frac{tx_mtu}{10}$ до $\frac{tx_mtu}{10}$, где tx_mtu - кол-во семплов, передаваемых за раз. Деление на 10 используется для масштабирования, чтобы не возводить большие числа в квадрат.

3.2.1 Программная реализация

```

int16_t* parabola_signal(int tx_mtu){

    // allocate memory
    int16_t* tx_buff = (int16_t*)malloc(sizeof(int16_t) * tx_mtu
        * 2);

    float coef = -tx_mtu / 10;

    for (int i = 0; i < 2 * tx_mtu; i+=2)
    {
        tx_buff[i] = int16_t(coef * coef);    // I
        tx_buff[i+1] = int16_t(coef * coef); // Q

        coef += 0.1;
    }

    return tx_buff;
}

```

3.3 Реализация BPSK модуляции

Шаги по реализации:

1. Написать BPSK модуляцию (логика маппера)
2. На основе I и Q, полученных на прошлом шаге, сгенерировать прямоугольные импульсы (логика формирующего фильтра)
3. Перемножить прямоугольные импульсы на несущее колебание
4. Проанализировать полученные результаты и сделать вывод

3.3.1 BPSK модулятор

```
double* BPSK_modulation(int* bits, int bits_count){
    //allocate memory
    double* IQ_samples = (double*)malloc(sizeof(double) *
        bits_count * 2);
    //iterate on bits
    for(int i,j = 0; i < bits_count * 2; i+=2){
        if(bits[j]){
            IQ_samples[i] = 1;        // I
            IQ_samples[i + 1] = 0;    // Q
        }else{
            IQ_samples[i] = -1;       // I
            IQ_samples[i + 1] = 0;    // Q
        }

        ++j;
    }

    return IQ_samples;
}
```

Сначала выделяем память под IQ семплы. Т.к на каждый бит приходится 1 семпл, а на 1 семпл - 2 числа (I и Q), то размер массива с семплами должен быть вдвое больше кол-ва бит. Далее перебираем биты и в зависимости от его значения формируем семпл. В результирующем массиве I и Q идут в строгой последовательности $I_0, Q_0, I_1, Q_1, \dots, I_N, Q_N$. Массив в дальнейшем запишется в файл.

3.3.2 Формирующий фильтр

На языке Python парсим файл с семплами из прошлого шага. Далее формируем прямоугольные импульсы.

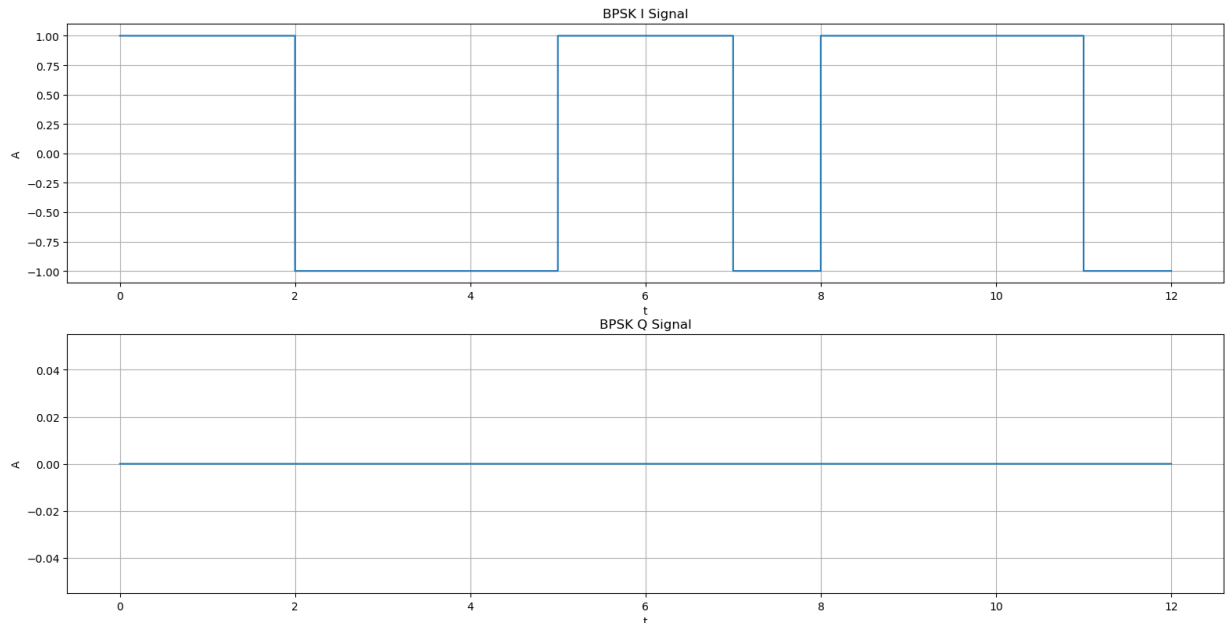


Рисунок 9 — BPSK-символы

Здесь явно видно, что в BPSK модуляции квадратурная составляющая всегда равна 0. Я передавал последовательность длиной 12 бит, и это заняло 12 единиц времени, т.е на каждый бит приходился 1 символ, который длился 1 единицу времени.

3.3.3 Перемножение с несущим колебанием

Теперь перемножим символы с несущим колебанием. В роли несущего колебания я использовал $\cos(2\pi ft)$ и $-\sin(2\pi ft)$ с частотой $f = 1$. В реальности частоты несущих колебаний в разы больше, но для наглядности я взял маленькое значение. Этот процесс можно сравнить с наложением маски, где маской является прямоугольный сигнал. Если посмотреть на схему цифровой системы связи, которая была дана в разделе с теорией, то этот процесс происходит в цепи, которая следует после формирующего фильтра.

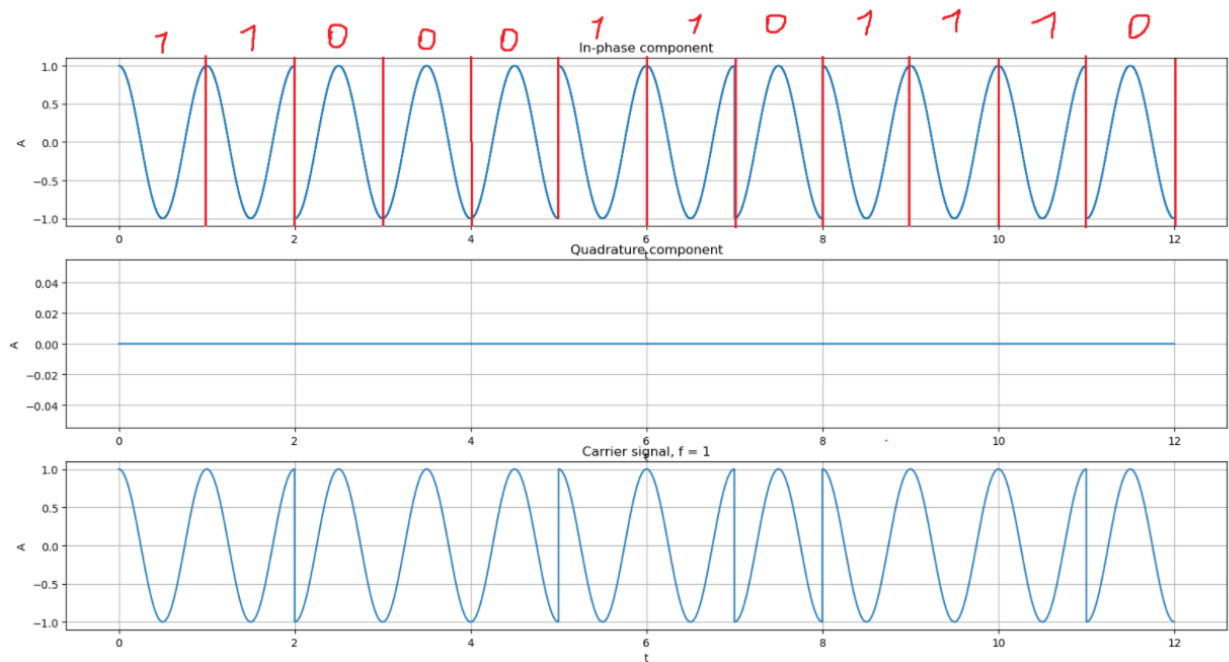


Рисунок 10 — Перемножение BPSK символов с несущим сигналом

Можем видеть, что квадратурная компонента по прежнему равна 0, т.е в сигнале присутствует только $\cos$. В синфазной компоненте можем видеть, как меняется фаза колебания, которая отражает значение бита. Если фаза равна 0, то передается 1, если фаза равна π , то передается 0. В местах, где меняется фаза, происходит смена значения бита. Таким образом можно восстановить передаваемую битовую последовательность. Результирующий сигнал будет иметь вид синфазной компоненты, т.к квадратурная всегда равна 0.

3.4 Реализация QPSK модуляции

Шаги по реализации будут в точности такие же, как у BPSK модуляции.

3.4.1 QPSK модулятор

```
double* QPSK_modulation(int* bits, int bits_count){
    //allocate memory
    double* IQ_samples = (double*)malloc(sizeof(double) *
        bits_count);
    //iterate on bits
    for(int i = 0; i < bits_count; i+=2){
        if(bits[i]){
```

```

        IQ_samples[i] = -1;           //I
        if(bits[i + 1]){
            IQ_samples[i + 1] = -1;   //Q
        }else{
            IQ_samples[i + 1] = 1;    // Q
        }
    }else{
        IQ_samples[i] = 1;           //I
        if(bits[i + 1]){
            IQ_samples[i + 1] = 1;    //Q
        }else{
            IQ_samples[i + 1] = -1;   //Q
        }
    }
}

return IQ_samples;
}

```

Заметим, что в случае QPSK модуляции массив под семплы вдвое меньше, т.к в QPSK модуляции на 1 семпл приходится 2 бита.

3.4.2 Формирующий фильтр

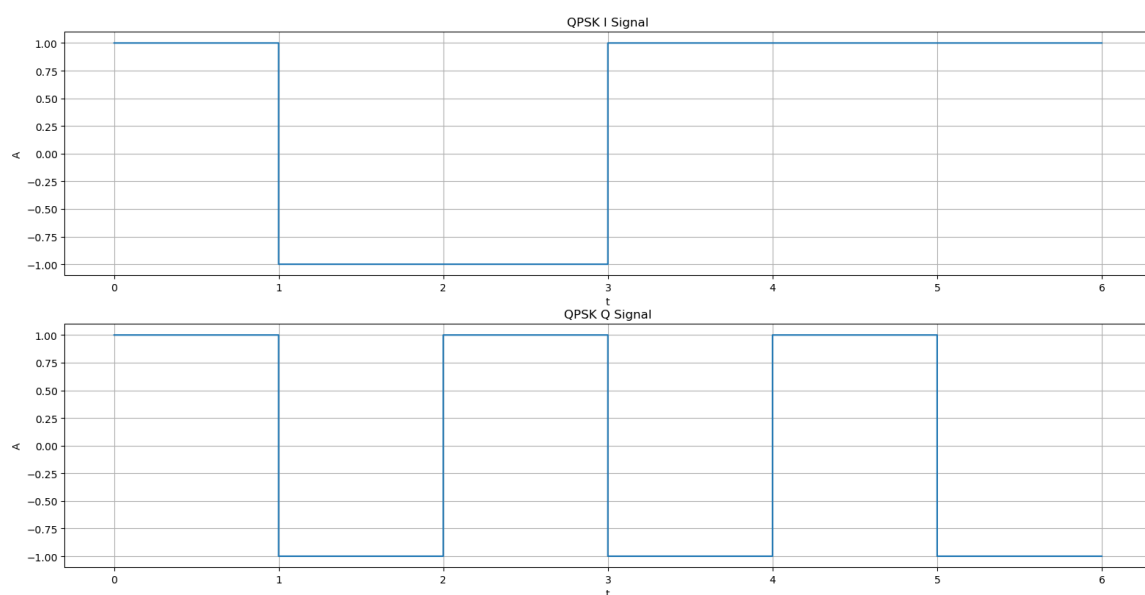


Рисунок 11 — QPSK символы

Последовательность из 12 бит передается уже за 6 единиц времени. Это связано с тем, что в QPSK на семпл приходится 2 бита. Можно сделать вывод о том, что чем больше точек в сигнальном созвездии, тем выше скорость передачи данных. 1 символ длится уже не 1 единицу времени, а 0.5 единиц времени (на графике чуть неверный масштаб)

3.4.3 Перемножение с несущим колебанием

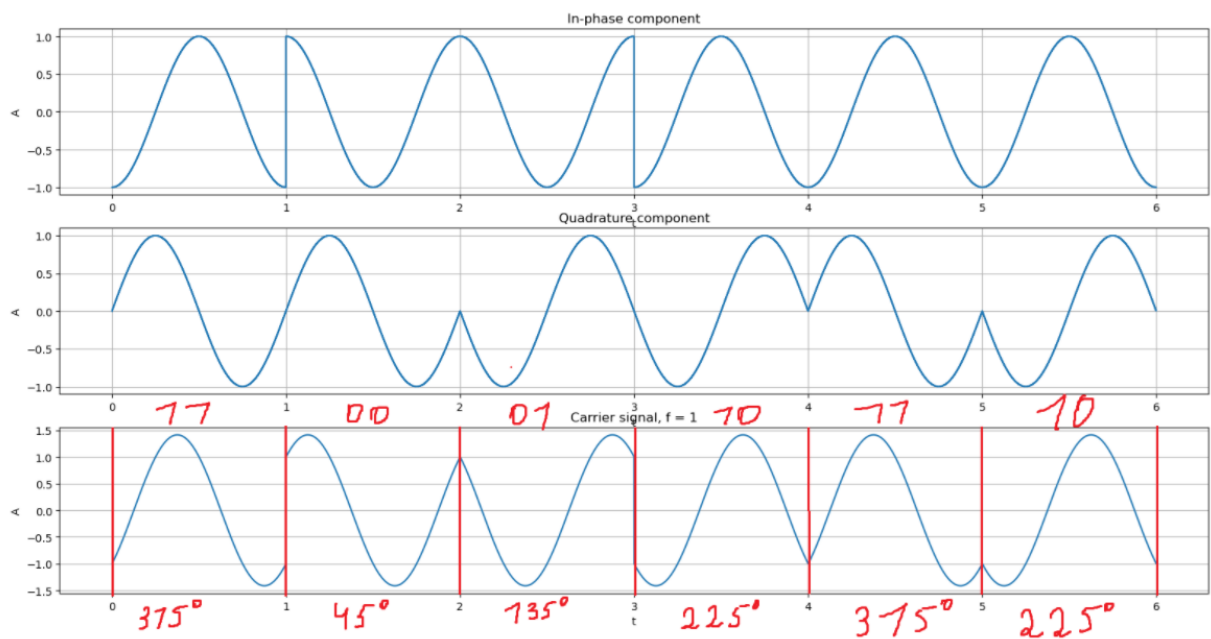


Рисунок 12 — Перемножение QPSK символов с несущим сигналом

Можем видеть, что квадратурная компонента теперь не равна 0, т.е в сигнале присутствует $\cos$ и $\sin$. Еще можно заметить, что максимальная амплитуда в результирующем сигнале уже не равна 1, а равна $\sqrt{I^2 + Q^2} = \sqrt{1^2 + 1^2} = \sqrt{2}$. Результирующий сигнал будет иметь вид $I\cos(2\pi f_c t) - Q\sin(2\pi f_c t)$.

3.5 Результат работы

3.5.1 Визуализация параболического сигнала до отправки

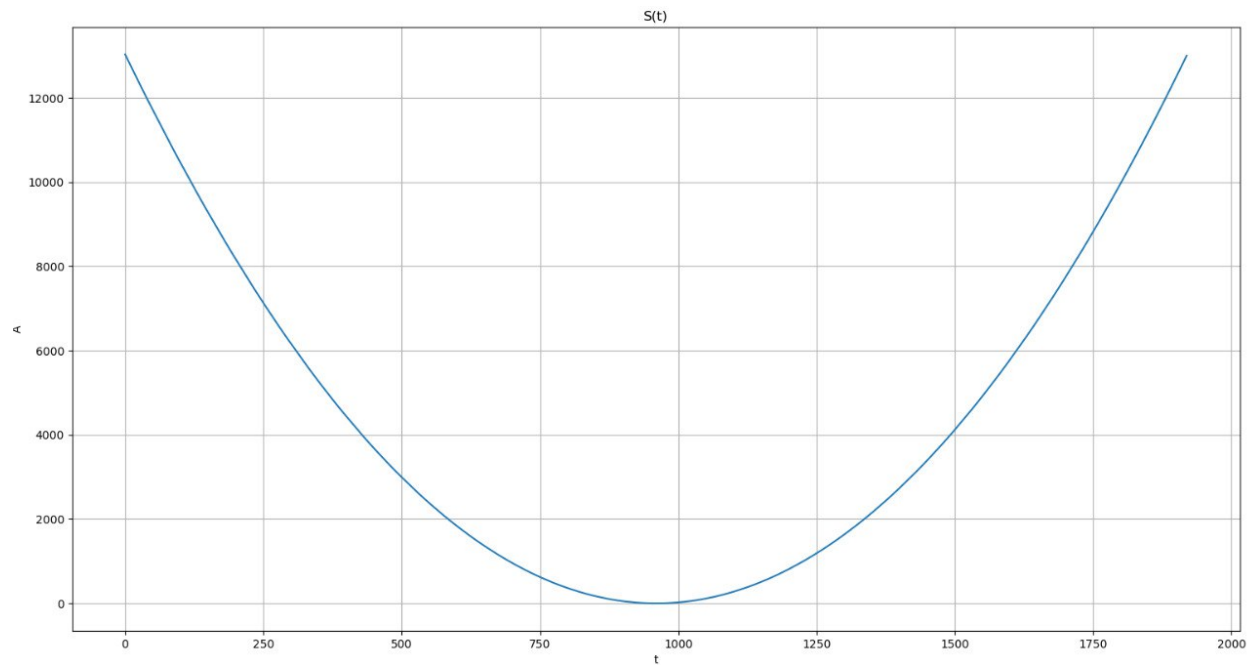


Рисунок 13 — Параболический сигнал до отправки

3.5.2 Визуализация параболического сигнала на приеме

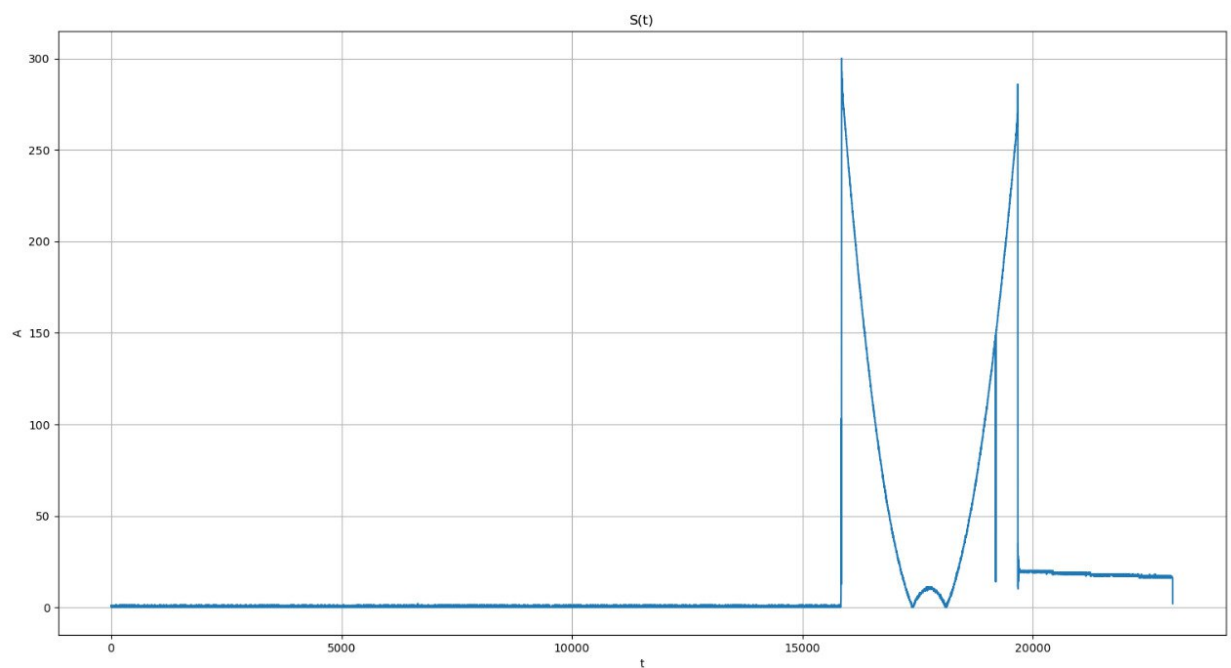


Рисунок 14 — Параболический сигнал на приеме

Можем наблюдать параболу в нашем сигнале, по центру параболы отображена вверх, потому что я строил график модуля сигнала.

3.5.3 Визуализация треугольного сигнала на приеме

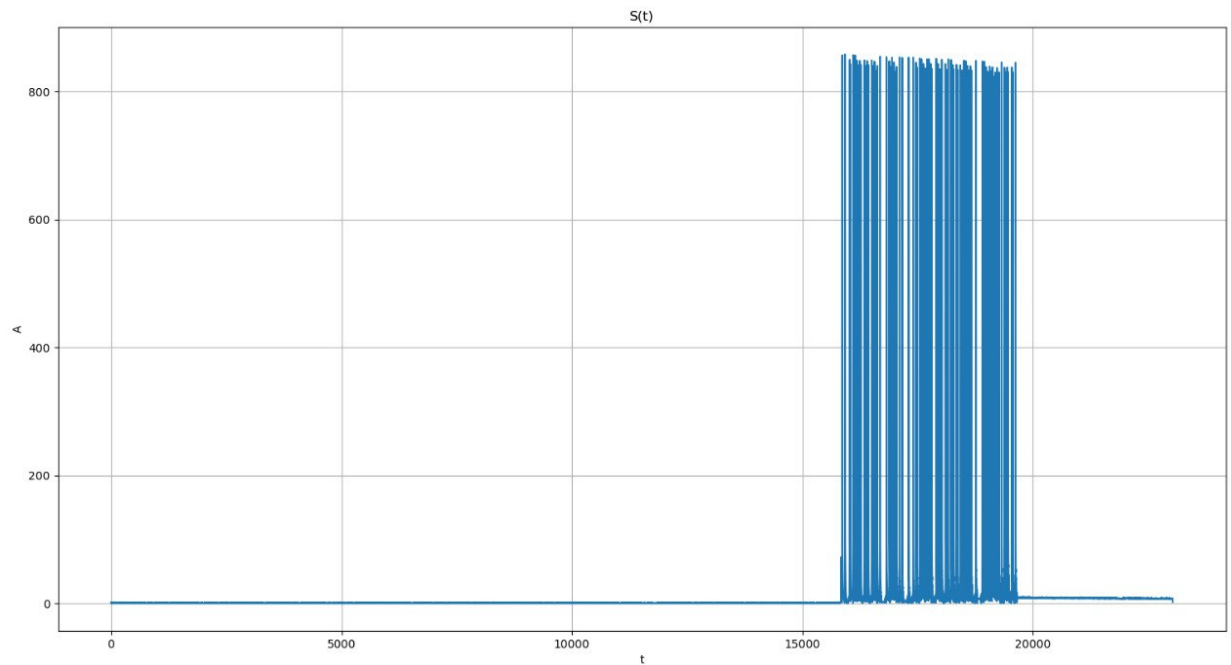


Рисунок 15 — Треугольный сигнал на приеме

ВЫВОД

В ходе проделанной работы я реализовал сигналы разной формы: параболической и треугольной, и визуализировал их. Также изучил базовые способы модуляции: BPSK и QPSK. На языке Python построил модель, которая подробно визуализирует процесс модуляции и перемножения с несущим колебанием.